

Introduction to Component Template

The template contains the HTML markup code to display the component in a web browser. This HTML never changes, unless it has bindings. Then it changes according to values in your component's class and becomes dynamic. Information about component's template is provided by the annotation. The template can be part of the components'.ts' file, or it can be in its own separate file. The code for the template markup Included in component annotation can be found at <http://dgit.in/TempMrkup>.

Introduction to Component Class

This TypeScript class contains both the data and the code for the Component. The data is contained in variables, which can be bound to the HTML markup in the template. Parent and Child Component:

1. Importing component object:

- `import { Component } from '@angular/core';`

ES6 style syntax is used by angular 2 which allows us to reuse by importing them from one file to other. In above code, Component object is imported from @angular/core.

2. Adding meta-data to class

- `@Component({`
- `selector: 'digit',`
- `template: '<h1>My First Subscription App</h1>'}))`

Meta data can be added to typescript using the @Component decorator.

Meta-data may include following:

- Selector:** It defines the name of the tag which is used to add component in html file.
- Styles:** It defines the css of the component. It takes an array of inline styles. To specify css(external) file urls, styleUrls are used instead of styles.
- Template:** Defines markup of the component. It takes a multiline or single line string. Multiline string can be specified using (`) backtick character. External templates can be specified by using templateUrl instead of template.
- Directives:** It helps in using nested components, it can be achieved by specifying the child component in the directives attribute. It takes an array which allows us to nest multiple child components inside the parent.
- Providers:** Services can be passed which are needed by the component using providers.